

1 Explainability in Reinforcement Learning

2 Maxime Alaarabiou

3 Monday 21st September, 2026

Chapter 1

Experiments

This chapter presents the experimental study of the Clustering Agent Representation (CAR) method introduced in the previous chapter. It focuses on evaluating the method’s ability to identify conceptual structures. The remainder of this chapter is organized as follows:

- Section 1.1 describes the software implementation of the CAR pipeline.
- Section 1.2 details the experimental configuration, covering the environments, the reinforcement learning setup, and the surrogate policy training.
- Section 1.3 presents the training of the initial and surrogate policies and evaluates how faithfully the surrogate policies reproduce the behavior of the initial policy.
- Section 1.4 provides a quantitative analysis of the latent representations using the *Clustering Universality* (CU), *Abstraction Score* (AS), and *Adjusted Clustering Universality* (ACU) metrics.
- Section 1.5 provides a qualitative analysis of the resulting clusters by manually examining their semantic content.
- Section 1.6 summarizes the findings and discusses their implications.

1.1 Software Implementation

This section details the experimental evaluation of the proposed method. Deep Learning (DL) and Reinforcement Learning (RL) are strongly empirical fields, where experimental evaluation plays an important role in assessing how methods perform in practice. Particular attention is therefore given to the implementation of CAR. It relies on established libraries that provide efficient implementations of its main components. The code is made publicly available as open-source to support reproducible replication of the reported results.¹

The CAR implementation is organized as a pipeline of five stages. Each stage performs a specific operation and produces the artifacts required by the subsequent stages. The stages are implemented as independent executables

¹https://github.com/Maxalaar/Clustering_Agent_Latent_Space



Figure 1.1: The CAR pipeline.

33 allowing each stage to be rerun independently. The CAR pipeline is illustrated in
 34 Figure 1.1, and Table 1.1 summarizes the libraries used throughout the pipeline.
 35 The remainder of this section describes the purpose and implementation of each
 36 stage.

37 1.1.1 Initial policy training

38 The first stage trains the initial policy using RLlib [Moritz et al., 2018, Liang
 39 et al., 2018]. RLlib provides scalable implementations of RL algorithms and
 40 supports distributed training. The use of RLlib also allows CAR to be applied to
 41 environments implemented using the Gymnasium interface [Towers et al., 2024].
 42 By default, the initial policy is trained using Proximal Policy Optimization
 43 (PPO) [Schulman et al., 2017]. Other RLlib algorithms, such as Asynchronous
 44 Proximal Policy Optimization (APPO) [Espeholt et al., 2018], Deep Q-Networks
 45 (DQN) [Mnih et al., 2013], and DreamerV3 [Hafner et al., 2023], can also be
 46 selected through the same configuration.

47 The policy and critic networks are implemented using PyTorch [Paszke et al.,
 48 2019]. PyTorch is the standard deep learning frameworks and is widely used for
 49 developing and training Neural Network (NN). It provides GPU acceleration
 50 and supports a wide range of NN architectures. This makes it well suited for
 51 implementing the different policy and critic architectures used in the experiments.

Pipeline Stage	Library	Description
Initial policy training	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed RL library used to train the initial policy.
	PyTorch [Paszke et al., 2019]	Deep learning framework implementing the policy and critic networks.
	Gymnasium [Towers et al., 2024]	Common interface used to plug in any RL environment.
Trajectory dataset generation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed environment runners used to collect trajectories in parallel.
	Apache Arrow [Apache Arrow, 2016]	Columnar file format (Parquet) used to store the trajectory datasets.
Surrogate policy training	PyTorch Lightning [Falcon, 2019]	Structured training loop used for the behavior cloning of the surrogate policies.
Surrogate policy evaluation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Wraps each surrogate policy as an RL policy to evaluate it in the environment.
Latent space analysis	cuML [RAPIDS, 2023]	GPU-accelerated clustering algorithms used to analyze the latent representations.

Table 1.1: Main software libraries used at each stage of the CAR pipeline.

1.1.2 Trajectory dataset generation

The second stage builds the dataset used to train the surrogate policies. RLlib is used to collect trajectories in parallel using multiple environment runners. For each step, the dataset stores the observation together with the parameters of the action distribution produced by the initial policy.

Trajectories are stored as Parquet files [Apache Arrow, 2016]. Its columnar structure allows efficient data access and supports concurrent processing. This is useful here because the datasets contain around one million observation–action pairs and cannot be loaded entirely into memory.

1.1.3 Surrogate policy training

The third stage trains the surrogate policies using behavior cloning on the trajectory dataset. This provides a supervised learning approach for replicating the initial policy. The training procedure is implemented using PyTorch Lightning [Falcon, 2019]. PyTorch Lightning provides a simplified interface for training NN with PyTorch. It handles common training operations such as data loading, optimization, checkpointing, and GPU execution.

1.1.4 Surrogate policy evaluation

The fourth stage evaluates how faithfully the surrogate policies reproduce the behavior of the initial policy. Each surrogate is wrapped as an RLlib policy and evaluated in the original environment. Its mean cumulative reward over multiple episodes is compared with that of the initial policy. Evaluation episodes are collected using parallel environment runners.

This stage computes the *Behavior Reconstruction*. It therefore verifies that the surrogate policies reproduce the behavior of the initial policy sufficiently well before their latent representations are analyzed. If a surrogate does not meet the required level of behavior reconstruction, its training can be repeated.

1.1.5 Latent space analysis

The fifth stage performs the main analysis of CAR. The trajectory dataset and trained surrogate policies are used to generate the latent representations. These representations form the point clouds that are analyzed using clustering algorithms.

The clustering step relies on the cuML library [RAPIDS, 2023], which provides GPU-accelerated clustering algorithms. This reduces the computational cost and allows point clouds containing tens of thousands of representations to be analyzed.

The resulting partitions are used to compute the *Clustering Universality* and the *Abstraction Score*. The stage also produces the material required for the qualitative analysis, including the clustered point clouds and the cluster assignment of each observation.

1.2 Experimental Configuration

This section describes the configuration of the experiments reported in this chapter. It covers the environments, the neural architectures used, and the main parameters of the pipeline.

1.2.1 Environments

All experiments presented in this manuscript use environments from the Atari benchmark [Bellemare et al., 2013], accessed through the Arcade Learning Environment. The use of Atari environments offers several advantages:

- **Standard benchmark:** Atari is widely used in the RL community, making the experimental results easier to compare with existing work.
- **Limited environment-specific bias:** The environments were not specifically designed for RL, reducing the risk of introducing biases related to their design.
- **Diversity:** The benchmark provides a diverse set of well-known environments with different types of tasks and levels of complexity, allowing CAR to be evaluated under different conditions.

The configuration of the Atari environments follows common practice in RL research [Machado et al., 2017]. In each environment, the agent receives visual observations in the form of raw pixels. Each image is resized to 64×64 pixels and converted to grayscale to reduce memory requirements. To provide information about the dynamics of the environment, four consecutive observations are stacked along the channel dimension. The resulting observation has a shape of $64 \times 64 \times 4$. This allows the agent to infer the motion of objects from successive observations.

The actions correspond to the different buttons available on the Atari controller. The resulting action space is therefore discrete. Since Atari games are fully deterministic, sticky actions are introduced to add stochasticity to the trajectories. With a probability of 0.05, the environment repeats the action selected at the previous step instead of applying the new action. This prevents the agent from simply memorizing a fixed sequence of actions during training.

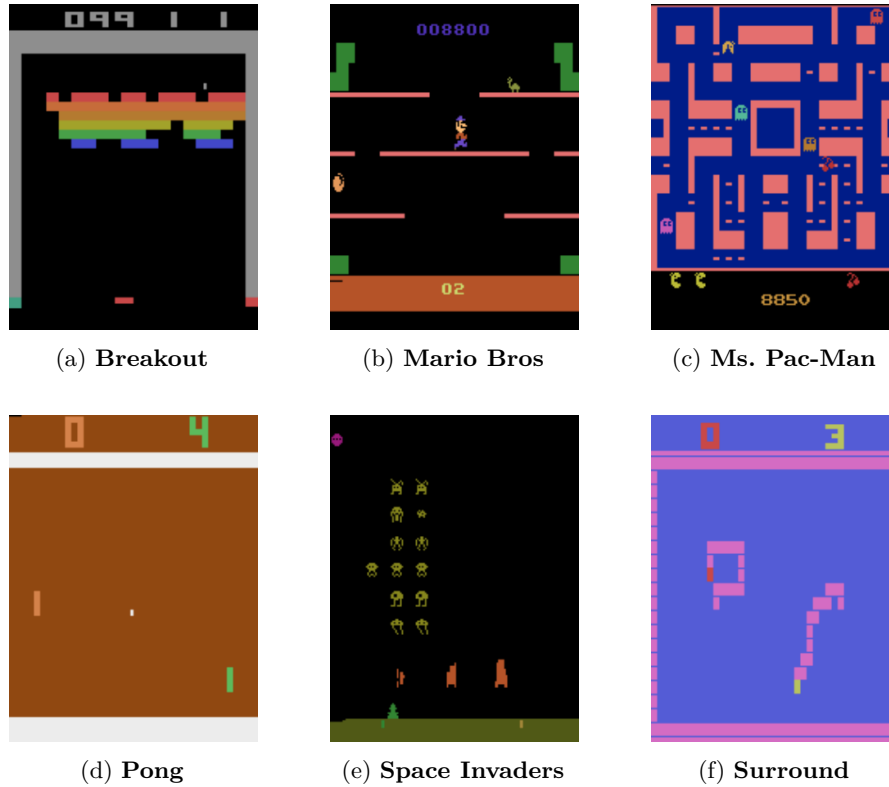


Figure 1.2: The six Atari environments used to evaluate CAR.

To reduce the duration of each episode, frame skipping is used. The agent selects a new action every five frames and repeats it for the following frames. This reduces the number of decisions required during an episode while maintaining sufficient temporal resolution for effective control.

The experiments use the six environments described below:

- **Breakout:** The agent moves a paddle at the bottom of the screen (Figure 1.2a) to bounce a ball into a wall of bricks. Each broken brick scores one point, and the ball speeds up as the wall is cleared.
- **Mario Bros.:** A single-screen platform game (Figure 1.2b) in which the player clears each phase by hitting enemies from below to flip them, then kicking them off the platforms before they recover. A central POW block can flip all enemies at once but can only be used a few times per phase.
- **Ms. Pac-Man:** The player moves through a maze (Figure 1.2c), eating pellets while avoiding four ghosts. Power pellets temporarily make the ghosts edible.
- **Pong:** A two-player game (Figure 1.2d) in which the agent controls one paddle and must send the ball past the opponent.
- **Space Invaders:** The agent moves a cannon along the bottom of the screen (Figure 1.2e), firing at descending rows of aliens while avoiding their



Figure 1.3: Architecture of the surrogate policies.

shots and using destructible shields. A bonus saucer occasionally crosses the top of the screen.

- **Surround:** A two-player game on a grid (Figure 1.2f) in which each player leaves a trail behind a moving cursor. The goal is to force the opponent to crash into a trail.

1.2.2 Neural architecture

The surrogate policies use a convolutional NN to process the visual observations. The convolutional part consists of four layers with 16, 32, 64, and 128 channels, using 4×4 kernels with a stride of 2. Each convolutional layer is followed by a LeakyReLU activation. The resulting representation is then flattened and passed through a multilayer perceptron that produces the parameters of the action distribution.

The architecture of the multilayer perceptron varies across environments. For **Pong** and **Space Invaders**, it contains a single hidden layer with 128 units. For the other four environments, it contains three hidden layers with 512, 256, and 128 units. The size of the output layer matches the number of actions available in each environment: 4 for **Breakout**, 18 for **Mario Bros**, 9 for **Ms. Pac-Man**, 6 for **Pong**, 6 for **Space Invaders**, and 5 for **Surround**.

Figure 1.3 illustrates both configurations, together with the shape of the representation produced by each convolutional layer. Throughout the remainder of this chapter, the analyzed representations are numbered from 1 to 5. These representations correspond to the four convolutional layers and the flattened representation, which are common to all six environments.

The representations produced by the multilayer perceptron are numbered separately. For **Pong** and **Space Invaders**, which use a single hidden layer, the representation is numbered 6a. For the other four environments, which use three hidden layers, the representations are numbered 6b, 7b, and 8b.

1.2.3 Other CAR parameters

Several other parameters must be defined for the different stages of the CAR pipeline. These parameters are less critical to the overall methodology than those discussed above, but they still need to be specified to reproduce the experiments.

Environment configuration	
Frame size	64×64
Frames stacked per observation	4
Frame skip	5
Sticky-action probability	0.05
Initial policy training	
Algorithm	PPO
Optimizer	Adam
Learning rate	1.5×10^{-4}
GAE λ	0.95
Policy clipping ε	0.1
Value-function clipping	10
Entropy coefficient	0.01
KL coefficient	0.5
Epochs per update	10
Gradient clipping (global norm)	100
Surrogate policy training	
Surrogate policies per environment	3
Observation–action pairs per dataset	$\geq 10^6$
Batch size	2048
Optimizer	Adam
Learning rate	1×10^{-4}
Latent space analysis	
Embedding per surrogate policy	20 000
Clustering algorithm	k -means
Distance	Euclidean distance
Number of clusters K	2 to 20

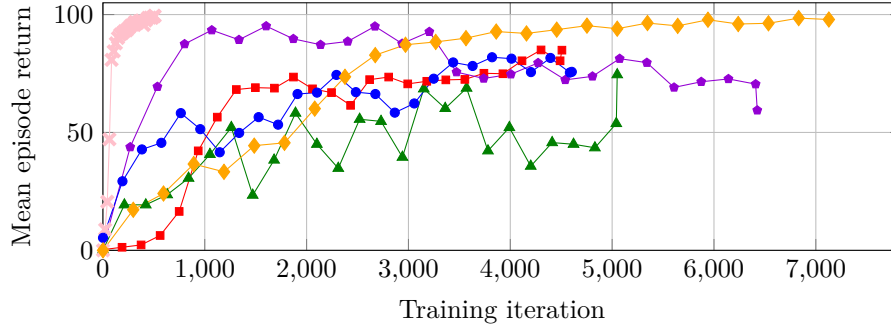
Table 1.2: Main experimental parameters.

Since each stage introduces its own configuration choices, there are too many parameters to discuss them all in detail. The main parameters used across the experiments are summarized in Table 1.2.

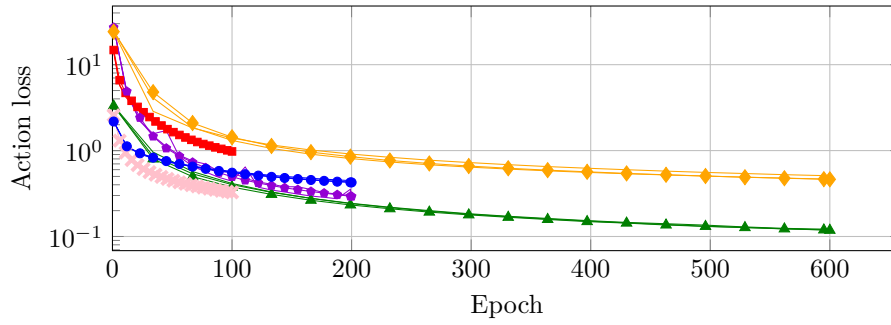
These settings are specific to the experiments presented in this chapter and do not restrict the CAR methodology itself. The different stages of the pipeline are modular and can be configured or replaced depending on the application. The evaluation metrics provide an objective basis for comparing these alternatives and assessing their effect on the consistency and meaningfulness of the resulting clusters.

1.3 Policy Training Analysis

This section reports the results of training the initial and surrogate policies for CAR, applied to the six environments. Section 1.3.1 verifies that the initial policies reach a competent level of play, while Section 1.3.2 evaluates whether the surrogate policies faithfully reproduce their behavior using the *Behavior*



(a) Training of the initial policies with PPO.



(b) Training action loss of the surrogate policies.



Figure 1.4: Training of the initial and surrogate policies.

184 *Reconstruction.*

185 1.3.1 Initial policy training

186 Figure 1.4a shows how the initial policies improve during PPO training. For
 187 each environment, the mean episode return is normalized so that 0% and 100%
 188 correspond to the lowest and highest mean return observed during that run. In
 189 every environment the return increases and then reaches a plateau, at which
 190 point training is stopped. The number of iterations required to reach this plateau
 191 varies from a few hundred, for **Pong**, to several thousand, for **Surround** and
 192 **Ms. Pac-Man**. For the subsequent analysis, the checkpoint with the best
 193 performance is retained rather than the final checkpoint.

194 To confirm that the trained policies play competently, their scores are com-
 195 pared with the reference scores reported by [Machado et al., 2017]. Each initial
 196 policy reaches a score of the same order as these references, so it constitutes a
 197 reasonable target for imitation.

198 1.3.2 Surrogate policy training

199 Figure 1.4b shows, for each environment, the training action loss of the three
 200 surrogate policies. In all cases, the loss decreases smoothly, and the three

Environment	<i>Behavior Reconstruction</i>
Breakout	0.83 ± 0.03
Mario Bros	0.99 ± 0.03
Ms. Pac-Man	1.00 ± 0.03
Pong	0.95 ± 0.01
Space Invaders	0.97 ± 0.07
Surround	1.00 ± 0.01

Table 1.3: Evaluation of *Behavior Reconstruction* across the six Atari environments.

independently trained policies converge to the same value.

We further evaluate the ability of the surrogate policies to replicate the behavior of the initial policies using the *Behavior Reconstruction* (Table 1.3). The surrogate policies reach a *Behavior Reconstruction* of 0.96 on average over 100 evaluation episodes, ranging from 0.83 for **Breakout** to 1.00 for **Ms. Pac-Man** and **Surround**. They therefore reproduce the decisions of the initial policies closely enough for their latent representations to be analyzed.

1.4 Quantitative Analysis

This section analyzes the Latent Space (LS) of the surrogate policies using CU, AS, and ACU. To do so, a random subset of 20 000 observations is selected for each environment. This subset is then embedded into the LS of each surrogate policy at each analyzed layer. This produces a point cloud for each combination of environment, surrogate policy, and layer. Each point cloud is partitioned using *k*-means clustering [MacQueen, 1967] for each number of clusters from 2 to 20.

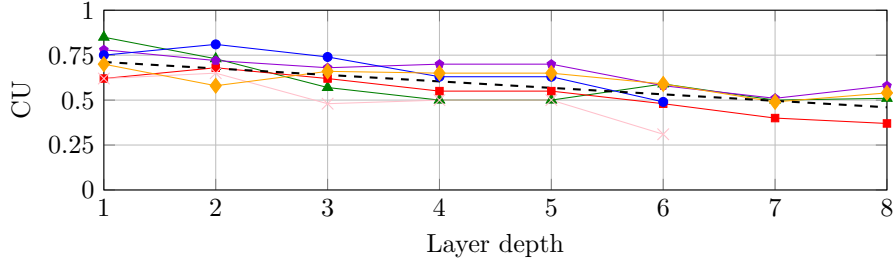
The three metrics are then computed from these point clouds and their corresponding clusterings. The full results across all environments and analyzed layers are reported in Appendix ???. Two complementary analyses are then conducted on these results. First, the evolution of the three metrics across the different layers is examined to study the effect of representation depth. Second, for each environment, the layer achieving the highest average ACU is selected and analyzed in more detail across the different numbers of clusters.

1.4.1 Analysis across layers

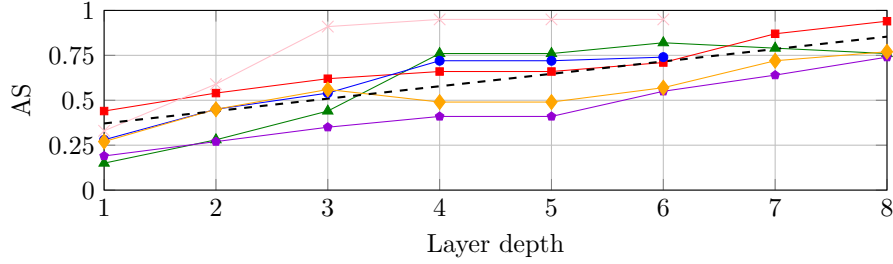
The first analysis examines how the three metrics evolve with the depth of the analyzed layer. For each layer, CU, AS, and ACU are averaged over the tested cluster numbers. Figure 1.5 reports these averages as a function of layer depth.

Across environments, CU tends to be highest in the earliest layers and decreases with depth. AS follows the opposite trend and increases as the representation becomes less tied to the raw observation, the action distribution, and the network’s initialization. As ACU combines these two effects, its evolution depends on the environment.

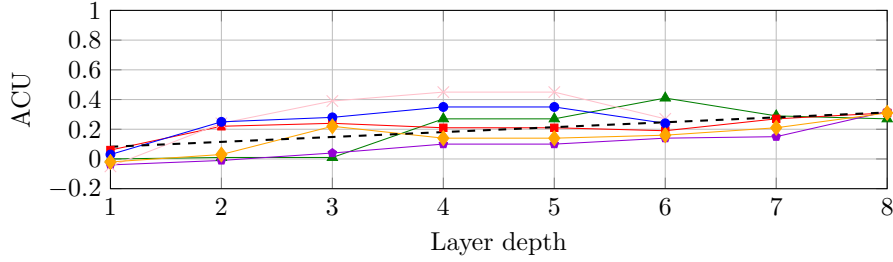
The corresponding linear fits show that CU decreases from 0.72 at the first layer to 0.50 at the last, with a slope of approximately -0.04 per layer, while AS increases from 0.28 to 0.80, with a slope of approximately $+0.07$ per layer. Over



(a) CU with respect to the layer depth.



(b) AS with respect to the layer depth.



(c) ACU with respect to the layer depth.



Figure 1.5: CU, AS, and ACU averaged over cluster numbers as a function of layer depth.

the same range, ACU increases more moderately from 0.00 to 0.30, with a slope of approximately +0.03 per layer. This suggests that, despite the environment-specific variations, deeper representations tend on average to provide a better trade-off between CU and AS.

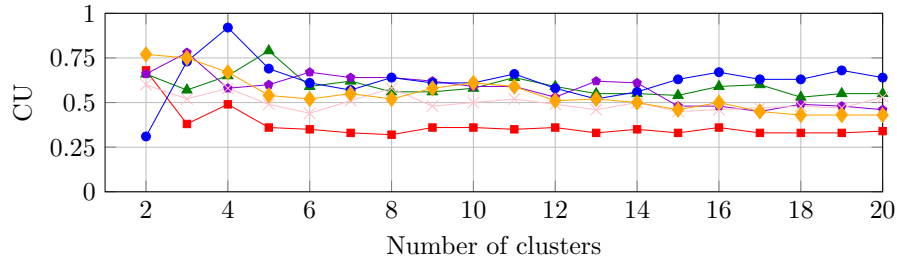
1.4.2 Analysis of the selected layers

The second analysis examines the effect of the number of clusters on the three metrics. For each environment, the layer with the highest average ACU across the tested cluster numbers is selected. Table 1.4 reports the selected layer, together with the corresponding average CU, AS, and ACU.

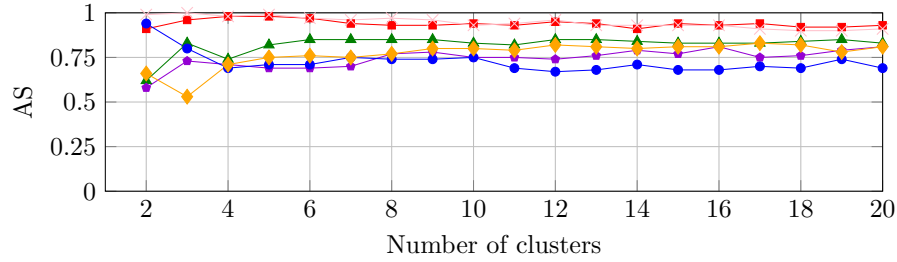
Figure 1.6 reports the evolution of CU, AS, and ACU as the number of clusters varies. **Pong** achieves the highest average ACU among the six environments,

with a value of 0.45 ± 0.07 . This results from a moderate CU combined with an AS that saturates from the third layer onward. In contrast, **Mario Bros** achieves the highest AS, with 0.94 ± 0.02 , but also the lowest CU, with 0.37 ± 0.08 . The resulting ACU is therefore among the lowest across the six environments.

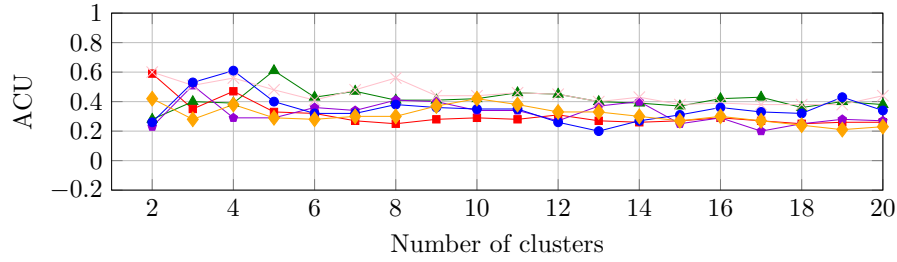
The quantitative analysis provides a measure of the consistency and abstraction of the identified clusters, but does not directly reveal their semantic content. The following section therefore provides a qualitative analysis of the clusters to examine the concepts they represent.



(a) CU with respect to the number of clusters.



(b) AS with respect to the number of clusters.



(c) ACU with respect to the number of clusters.

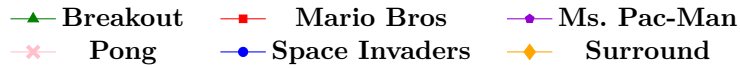


Figure 1.6: CU, AS, and ACU with respect to the number of clusters, for each of the six environments, for the layer that maximizes the average ACU in that environment.

Environment	Layer	CU	AS	ACU
Breakout	6b	0.59 $\pm$ 0.06	0.82 $\pm$ 0.05	0.41 $\pm$ 0.06
Mario Bros	8b	0.37 $\pm$ 0.08	0.94 $\pm$ 0.02	0.31 $\pm$ 0.08
Ms. Pac-Man	8b	0.58 $\pm$ 0.08	0.74 $\pm$ 0.05	0.32 $\pm$ 0.07
Pong	4	0.50 $\pm$ 0.04	0.95 $\pm$ 0.03	0.45 $\pm$ 0.07
Space Invaders	4	0.63 $\pm$ 0.11	0.73 $\pm$ 0.06	0.35 $\pm$ 0.09
Surround	8b	0.54 $\pm$ 0.10	0.77 $\pm$ 0.07	0.31 $\pm$ 0.06

Table 1.4: Layer that maximizes the average ACU across $K \in [2, 20]$ for each environment, together with the corresponding average CU, AS, and ACU over K .

1.5 Qualitative Analysis

The metrics reported in Section 1.4 evaluate the LS clusterings quantitatively but do not indicate what the resulting clusters actually represent. This section complements them with a qualitative, hand-labeled inspection of the clusters obtained for each environment.

For each environment, two clusterings are selected for the qualitative analysis. They are obtained using two different numbers of clusters and may correspond to different layers of the network. The choice of the layer and the number of clusters is based on the following criteria:

- **High ACU:** The selected clustering should achieve a sufficiently high ACU, indicating a good balance between consistency and abstraction.
- **Manual analysis:** The number of clusters must remain small enough to allow the resulting clusters to be examined manually.
- **Limited use of two-cluster partitions:** Clusterings with two clusters often achieve a high ACU, but provide limited information about the organization of the LS. Therefore, at most one of the two selected clusterings uses two clusters, while the other uses a larger number of clusters to provide a more detailed view of the latent space.

1.5.1 Breakout

Two clusterings are examined for **Breakout** (Table 1.5), obtained at layer 4 with $K = 2$ and at layer 6b with $K = 5$.

At layer 4 (Figure 1.7), the partition captures the position of the ball relative to the brick wall, with the ball below the wall in cluster A and having passed behind it in cluster B. The latter corresponds to the point at which the ball can continue clearing bricks without further interaction with the paddle.

At layer 6b (Figure 1.8), the partition also isolates the ball behind the wall in cluster E and the digging of a tunnel through the left side of the wall in cluster C. It also distinguishes different paddle positions at the beginning of the episode in clusters D, F, and G.

The clustering at layer 4 with $K = 2$ is almost perfectly reproducible across surrogate policies, with a CU of 0.99 and an ACU of 0.87. The clustering at layer 6b with $K = 5$ is less consistent, with a CU of 0.79 and an ACU of 0.61, but reveals additional information about the organization of the LS.

Layer 4	$K = 2$	CU 0.99	AS 0.88	ACU 0.87
A	The ball is below the brick wall.			
B	The ball passes behind the brick wall.			
Layer 6b	$K = 5$	CU 0.79	AS 0.82	ACU 0.61
C	Digging a tunnel through the left side of the wall.			
D	Start of the episode, paddle on the right.			
E	The ball is behind the brick wall.			
F	Start of the episode, paddle in the middle.			
G	Start of the episode, paddle on the left.			

Table 1.5: CU, AS, and ACU of the two clusterings analyzed in the **Breakout** environment.

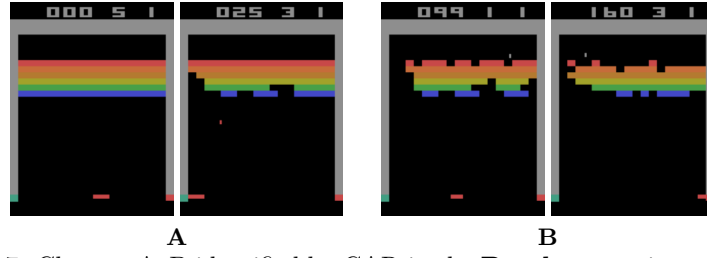


Figure 1.7: Clusters A–B identified by CAR in the **Breakout** environment (layer 4, $K = 2$).

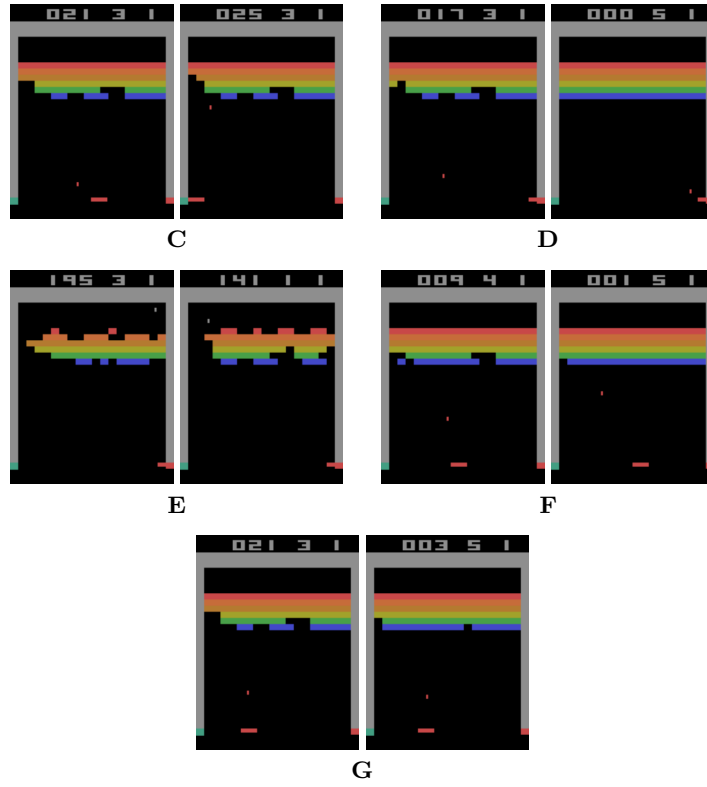


Figure 1.8: Clusters C–G identified by CAR in the **Breakout** environment (layer 6b, $K = 5$).

1.5.2 Mario Bros

Two clusterings are examined for **Mario Bros** (Table 1.6), both obtained at layer 3, with $K = 3$ and $K = 5$. Since the layer is the same, this comparison isolates the effect of K .

With $K = 3$ (Figure 1.9), the partition is organized around the location of the action. It separates the side of the screen on which Mario fights enemies, with the left side forming cluster A and the right side forming cluster C. It also isolates situations in which the POW block is present in cluster B, corresponding to a fixed and recognizable element of the level.

With $K = 5$ (Figure 1.10), the POW block is still isolated in cluster E, but the partition no longer separates the left and right sides of the screen. It distinguishes finer situations, mostly located on the left side of the level. These include Mario on the top-left platform in cluster D, attacking enemies at the top left in cluster F, and hitting enemies from below in cluster H. Cluster G corresponds to situations in which Mario moves through the level.

Increasing K from 3 to 5 lowers CU from 0.84 to 0.75 and leaves AS nearly unchanged at 0.66 and 0.65, resulting in a decrease in ACU from 0.50 to 0.40.

Layer 3	$K = 3$	CU 0.84	AS 0.66	ACU 0.50
A	Mario fights enemies on the left side of the screen.			
B	The POW block is present.			
C	Mario fights enemies on the right side of the screen.			
Layer 3	$K = 5$	CU 0.75	AS 0.65	ACU 0.40
D	Mario is on the top-left platform.			
E	The POW block is present.			
F	Mario attacks the enemies at the top right.			
G	Mario moves through the level.			
H	Mario hits enemies from below on the left side.			

Table 1.6: CU, AS, and ACU of the two clusterings analyzed in the **Mario Bros** environment.

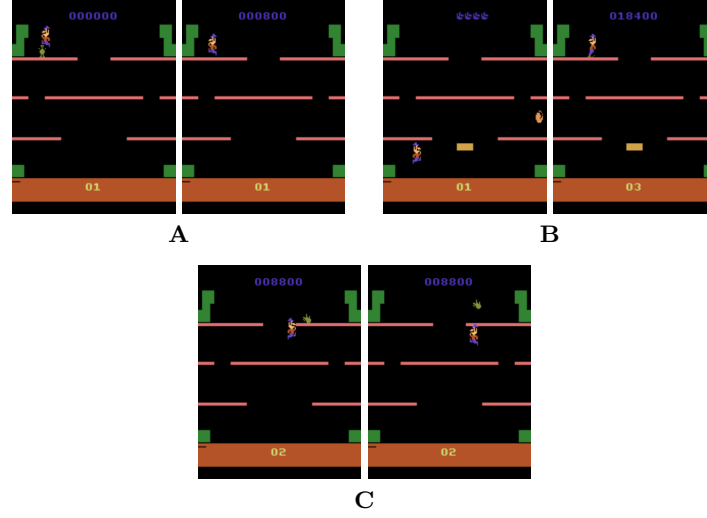


Figure 1.9: Clusters A–C identified by CAR in the **Mario Bros** environment (layer 3, $K = 3$).

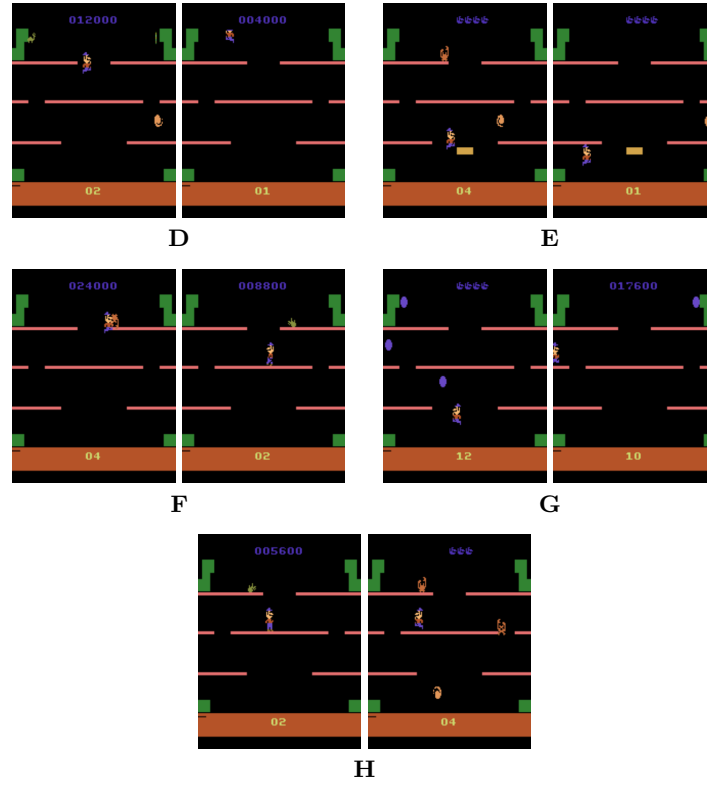


Figure 1.10: Clusters D–H identified by CAR in the **Mario Bros** environment (layer 3, $K = 5$).

1.5.3 Ms. Pac-Man

Two clusterings are examined for **Ms. Pac-Man** (Table 1.7), obtained at layer 6b with $K = 3$ and at layer 4 with $K = 5$. Both clusterings capture the progress of the episode and the state of the game, rather than a specific spatial feature of the observation as observed in **Mario Bros**.

With $K = 3$ (Figure 1.11), the partition separates the start of the episode in cluster B, the middle in cluster A, and the end in cluster C.

With $K = 5$ (Figure 1.12), the partition provides a more detailed description of the episode. It separates the initial animation, during which the character cannot move, into cluster H and the beginning of actual play into cluster F. It also isolates a situation specific to the game mechanics, in which Pac-Man is in booster mode and waits in front of the ghost spawn to eat the ghosts in cluster D. Finally, it separates the end of the episode, when at most one power pellet remains, into two similar clusters, E and G.

The two clusterings have the same CU of 0.88, while the clustering at layer 4 has a lower AS of 0.47 compared with 0.52 at layer 6b. This results in a lower ACU of 0.35 compared with 0.40.

Layer 6b	$K = 3$	CU 0.88	AS 0.52	ACU 0.40
A	Middle of the episode.			
B	Start of the episode.			
C	End of the episode.			
Layer 4	$K = 5$	CU 0.88	AS 0.47	ACU 0.35
D	Pac-Man is in booster mode and waits in front of the ghost spawn to eat the ghosts.			
E	End of the episode, at most one power pellet remains (similar to cluster G).			
F	Start of the episode.			
G	End of the episode, at most one power pellet remains (similar to cluster E).			
H	Start-of-episode animation, the character cannot move.			

Table 1.7: CU, AS, and ACU of the two clusterings analyzed in the **Ms. Pac-Man** environment.

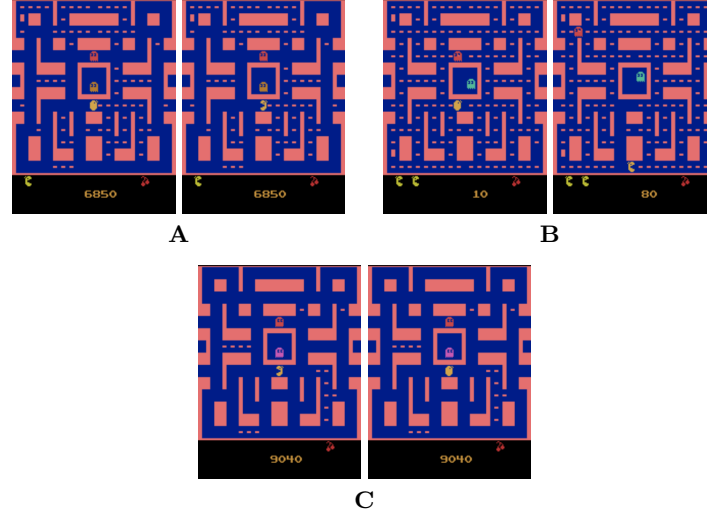


Figure 1.11: Clusters A–C identified by CAR in the **Ms. Pac-Man** environment (layer 6b, $K = 3$).

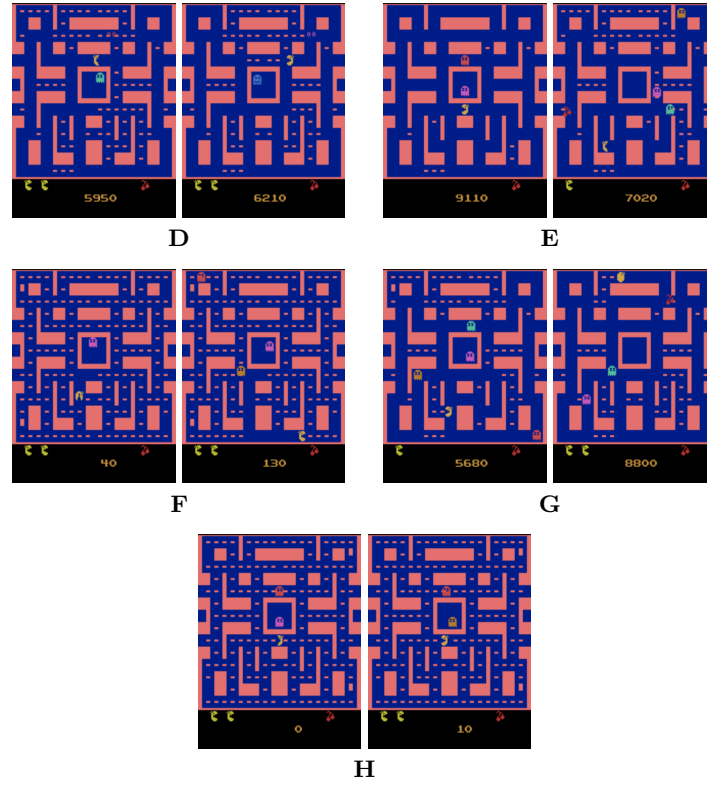


Figure 1.12: Clusters D–H identified by CAR in the **Ms. Pac-Man** environment (layer 4, $K = 5$).

1.5.4 Pong

Two clusterings are examined for **Pong** (Table 1.8), obtained at layer 2 with $K = 2$ and at layer 3 with $K = 3$.

At layer 2 (Figure 1.13), the partition separates the observations according to whether the player's score has one or two digits (clusters A and B). This reflects the progress of the game, but only through its visual display at the top of the screen. The partition therefore captures a feature of the observation that provides no information about how the agent plays.

At layer 3 (Figure 1.14), the partition instead follows the vertical position of the ball. It distinguishes situations in which the ball is at the top (cluster C), in the middle (cluster D), or at the bottom (cluster E) of the field. This feature directly determines where the agent needs to place its paddle.

The clustering at layer 2 is perfectly reproducible and perfectly abstract, with CU, AS, and ACU all equal to 1.00. The clustering at layer 3 is less consistent, with a CU of 0.67, and reaches a lower ACU of 0.65.

Layer 2	$K = 2$	CU 1.00	AS 1.00	ACU 1.00
A	The player's score has two digits.			
B	The player's score has one digit.			
Layer 3	$K = 3$	CU 0.67	AS 0.98	ACU 0.65
C	The ball is at the top.			
D	The ball is in the middle.			
E	The ball is at the bottom.			

Table 1.8: CU, AS, and ACU of the two clusterings analyzed in the **Pong** environment.

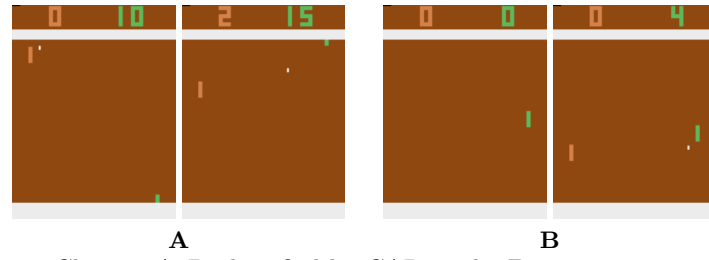


Figure 1.13: Clusters A–B identified by CAR in the **Pong** environment (layer 2, $K = 2$).

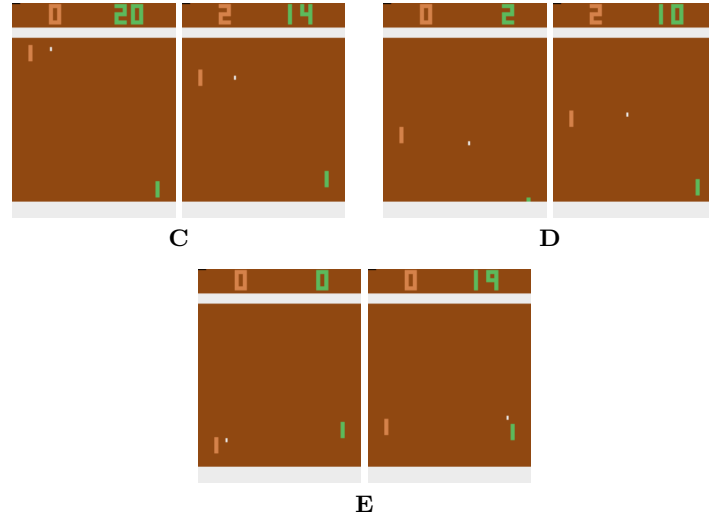


Figure 1.14: Clusters C–E identified by CAR in the **Pong** environment (layer 3, $K = 3$).

1.5.5 Space Invaders

Two clusterings are examined for **Space Invaders** (Table 1.9), obtained at layer 4 with $K = 4$ and at layer 2 with $K = 5$.

At layer 4 (Figure 1.15), three clusters track the progression of the episode, from the start (cluster D) to the middle (cluster B) and the end (cluster C). The fourth cluster isolates a behavior in which the agent has cleared the leftmost column and is positioned to catch the flying saucer that occasionally crosses the top of the screen (cluster A).

At layer 2 (Figure 1.16), three clusters again track the progression of the episode, from the start (cluster G) to the middle (cluster F) and the end (cluster I). The behavior is now split between attacking the first two columns of the formation (cluster E) and trying to catch the flying saucer (cluster H).

The two clusterings are equally consistent across surrogate policies, with a CU of 0.92 in both cases. The clustering at layer 2 is less abstract, with a AS of 0.49 compared with 0.69 at layer 4, resulting in a lower ACU of 0.41 compared with 0.61.

Layer 4	$K = 4$	CU 0.92	AS 0.69	ACU 0.61
A	The left column has been cleared; the flying saucer occasionally appears.			
B	Middle of the episode.			
C	End of the episode.			
D	Start of the episode.			
Layer 2	$K = 5$	CU 0.92	AS 0.49	ACU 0.41
E	Attacks the first two columns of the formation.			
F	Middle of the episode.			
G	Start of the episode.			
H	Tries to catch the flying saucer.			
I	End of the episode.			

Table 1.9: CU, AS, and ACU of the two clusterings analyzed in the **Space Invaders** environment.

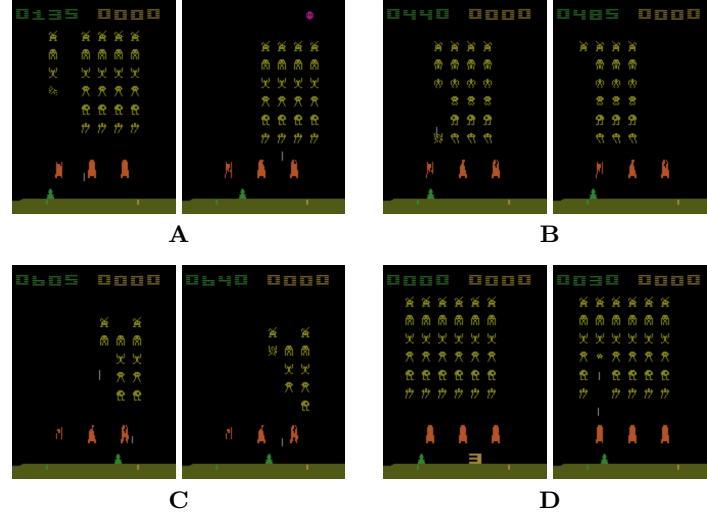


Figure 1.15: Clusters A–D identified by CAR in the **Space Invaders** environment (layer 4, $K = 4$).

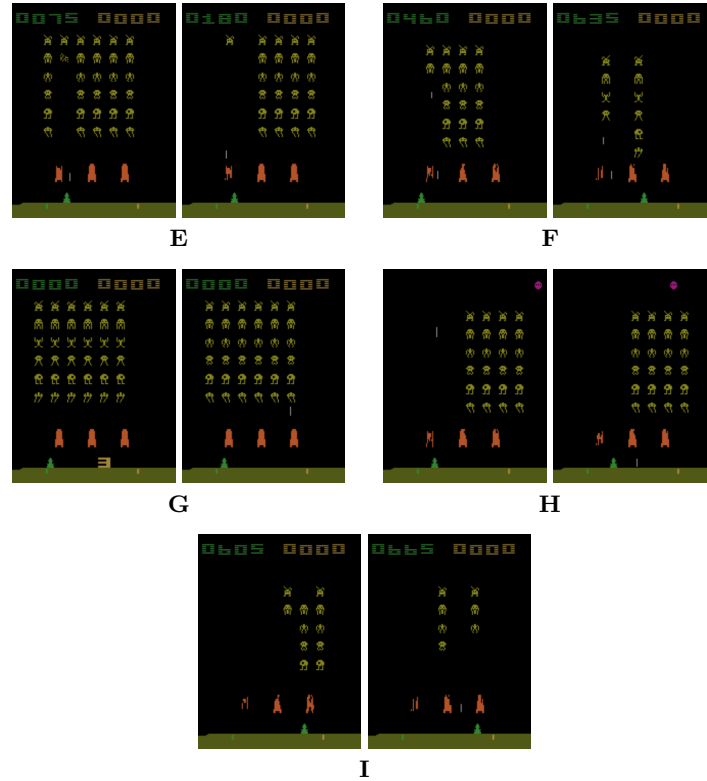


Figure 1.16: Clusters E–I identified by CAR in the **Space Invaders** environment (layer 2, $K = 5$).

1.5.6 Surround

Two clusterings are examined for **Surround** (Table 1.10), obtained at layer 8b with $K = 4$ and at layer 7b with $K = 6$.

At layer 8b (Figure 1.17), the partition combines three types of concepts. The start of the episode is captured by cluster B, the outcome of the episode, when the opponent loses, by cluster D, and the region of the level in which the player moves by clusters A and C. This region is described coarsely, with the player's trail either confined to the bottom-left area (cluster A) or running along the top of the level after arriving from the left (cluster C).

At layer 7b (Figure 1.18), the partition again captures the start of the episode (cluster J) and its outcome, when the opponent loses (cluster F). The region of the level is described in greater detail, with the player moving in the upper-left corner (cluster E) or in the lower-left corner (cluster G). The end of the game is also isolated in the lower-left corner (cluster H), while cluster I captures situations in which the player crosses the level from the top left to the bottom right.

Both clusterings reach the same CU, AS, and ACU, with values of 0.67, 0.71, and 0.38, respectively. The refinement therefore neither improves nor degrades the metrics.

Layer 8b	$K = 4$	CU 0.67	AS 0.71	ACU 0.38
A	The agent's trail remains confined to the bottom-left area.			
B	Start of the episode.			
C	The agent moves along the top of the level, arriving from the left.			
D	The opponent loses by running into a trail.			
Layer 7b	$K = 6$	CU 0.67	AS 0.71	ACU 0.38
E	The player moves in the upper-left corner.			
F	The opponent loses the confrontation.			
G	The player moves in the lower-left corner.			
H	The end of the game is played in the lower-left corner.			
I	The player starts at the top left and ends at the bottom right.			
J	Start of the episode.			

Table 1.10: CU, AS, and ACU of the two clusterings analyzed in the **Surround** environment.

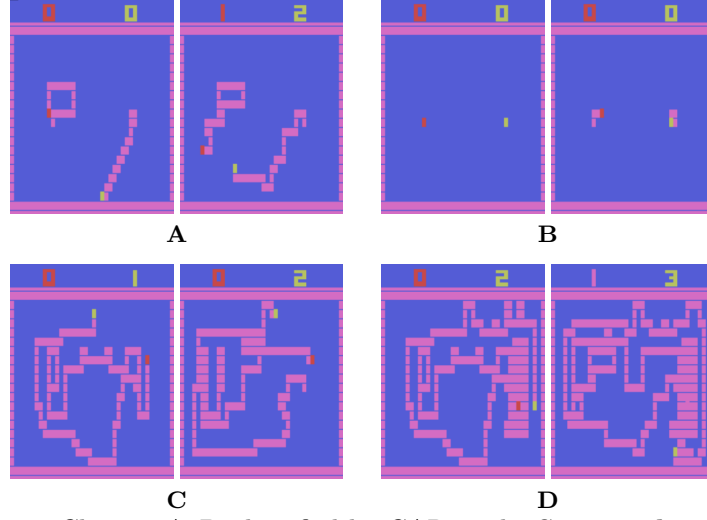


Figure 1.17: Clusters A–D identified by CAR in the **Surround** environment (layer 8b, $K = 4$).

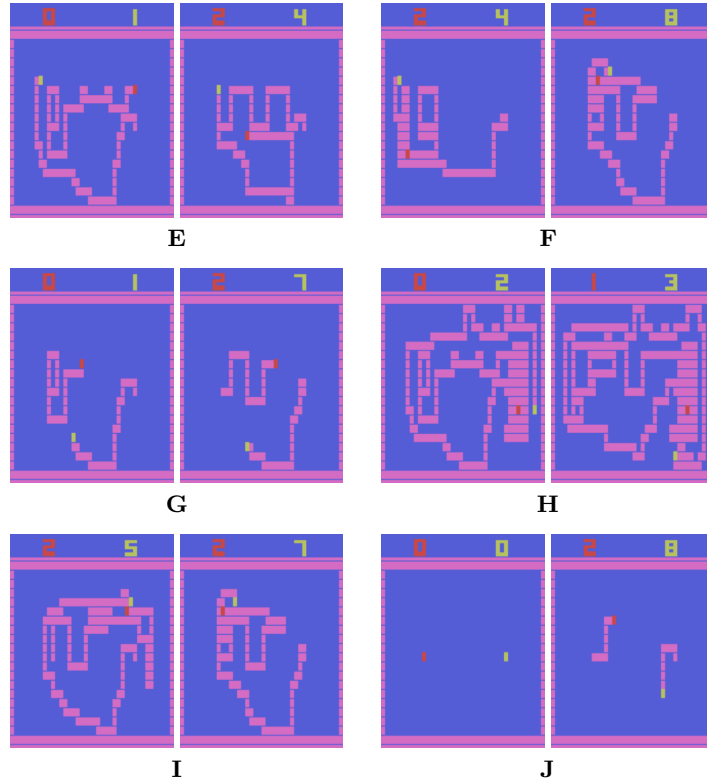


Figure 1.18: Clusters E–J identified by CAR in the **Surround** environment (layer 7b, $K = 6$).

1.6 Conclusion

The quantitative results show that CAR often recovers similar clusterings across independently trained surrogate policies. In many of the tested configurations, the CU exceeds 0.5, indicating that the latent representations developed by different policies contain sufficiently similar structures to produce comparable partitions [Amezquita et al., 2020]. This result supports the *Representation Convergence Hypothesis* (Hyp. ??), which assumes that independently trained networks tend to develop similar conceptual organizations when solving the same task. However, CU generally decreases with the depth of the network. The stability of the representations is therefore progressively reduced in deeper layers.

The AS provides an important complement to this observation. The earliest layers tend to achieve high CU, but also exhibit low levels of abstraction because their representations remain strongly related to the observations, the action distribution, and the initial network structure. Considering CU alone would therefore favor the earliest layers without necessarily identifying the most meaningful representations. The ACU addresses this limitation by combining consistency and abstraction into a single measure. It therefore provides a practical criterion for identifying regions of the latent space that offer a useful balance between these two properties. The metric is also fully automated and can be applied consistently across different environments and network configurations. Increasing the number of clusters generally reduces ACU, as finer partitions tend to be less stable across surrogate policies. At the same time, these finer partitions reveal more specific distinctions between situations encountered during the simulations. This highlights a trade-off between the stability of the recovered concepts and the level of detail captured by the clustering.

The qualitative analysis complements these quantitative results by showing that the identified clusters can correspond to interpretable properties of the environment and of the agent’s behavior. Across the different environments, clusters capture recurring aspects such as the progression of an episode, specific game situations, and patterns related to the agent’s behavior. Increasing the number of clusters generally produces a finer description of these situations, although the resulting distinctions are often less consistently recovered across surrogate policies. Together, these results support the assumptions underlying CAR and show that the proposed metrics provide a practical way to identify and compare meaningful latent-space organizations across independently trained policies.

407 Acronyms

- 408 **ACU** *Adjusted Clustering Universality* 1, 9–13, 15, 17, 19, 21, 23, 25
409 **APPO** Asynchronous Proximal Policy Optimization 2
410 **AS** *Abstraction Score* 1, 9–13, 15, 17, 19, 21, 23, 25
- 411 **CAR** Clustering Agent Representation 1–7, 14, 16, 18, 20, 22, 24, 25
412 **CU** *Clustering Universality* 1, 9–13, 15, 17, 19, 21, 23, 25
- 413 **DL** Deep Learning 1
414 **DQN** Deep Q-Networks 2
- 415 **GPU** Graphics Processing Unit 3
- 416 **LS** Latent Space 9, 12, 13
- 417 **NN** Neural Network 2, 3, 6
- 418 **PPO** Proximal Policy Optimization 2, 7, 8
- 419 **RL** Reinforcement Learning 1–4

Bibliography

- [Amezquita et al., 2020] Amezquita, R. A., Lun, A. T., Becht, E., Carey, V. J., Carpp, L. N., Geistlinger, L., Marini, F., Rue-Albrecht, K., Risso, D., Soneson, C., et al. (2020). Orchestrating single-cell analysis with Bioconductor, volume 17. Nature Publishing Group US New York. **Book**.
- [Apache Arrow, 2016] Apache Arrow (2016). Apache Arrow and Parquet. <https://arrow.apache.org>.
- [Bellemare et al., 2013] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research, 47:253–279. **Article**.
- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Falcon, 2019] Falcon, W. (2019). PyTorch Lightning. <https://github.com/Lightning-AI/pytorch-lightning>.
- [Hafner et al., 2023] Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. (2023). Mastering diverse domains through world models.
- [Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Machado et al., 2017] Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., and Bowling, M. (2017). Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. **Article. Web**.
- [MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, volume 1, pages 281–297. **Article**.
- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.

- 454 [Moritz et al., 2018] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R.,
455 Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018).
456 Ray: A distributed framework for emerging AI applications. In 13th USENIX
457 Symposium on Operating Systems Design and Implementation (OSDI), pages
458 561–577.
- 459 [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J.,
460 Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison,
461 A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy,
462 S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An
463 imperative style, high-performance deep learning library. In Advances in
464 Neural Information Processing Systems (NeurIPS).
- 465 [RAPIDS, 2023] RAPIDS (2023). RAPIDS cuML: GPU-accelerated machine
466 learning. <https://github.com/rapidsai/cuml>.
- 467 [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and
468 Klimov, O. (2017). Proximal policy optimization algorithms.
- 469 [Towers et al., 2024] Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U.,
470 De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG,
471 A., et al. (2024). Gymnasium: A standard interface for reinforcement learning
472 environments.